Kigali Shaolin Temple Management System

Frontend Website, Admin Portal (CMS), and Backend API

Purpose:
A unified professional blueprint combining system analysis, UI/UX design, CMS control, and backend logic.

Prepared for: Technical Documentation
Year: 2025

Contents

1 General Project Overview **3**

2 System Architecture **3**

3 Branding and UI Design System (Tailwind) **4**
3.1 Color Palette 4
3.2 Tailwind Configuration 4
3.3 Typography 4

4 Frontend Website (Public) **4**
4.1 Design Goals 4

5 Frontend Pages and Sections **5**
5.1 Header and Navigation 5
5.2 Hero Section 5
5.3 About Us Section 5
5.4 Programs and Training 6
5.5 Blogs 6
5.6 Clubs 6
5.7 Contact Us 6
5.8 Gallery 7

6 Admin Portal (CMS Dashboard) **7**
6.1 Authentication and Roles 7
6.2 Dashboard Modules 7

7 Backend (API and Logic) **8**
7.1 Core Responsibilities 8
7.2 Technology Stack 8

8 Database Models **8**

9 Advanced System Features **9**
9.1 Audit Logging 9
9.2 Data Recovery and Soft Deletes 9
9.3 Interactive API Documentation 9
9.4 ID Obfuscation (UUID Encryption) 9
9.5 Database Schema Standardization 10
9.6 Frontend Stability Enhancements 10

10 Non-Functional Requirements **11**

11 Final Summary **11**

1 General Project Overview

Project Name: Kigali Shaolin Temple Management System

Purpose: To promote Shaolin culture in Rwanda, manage clubs, programs, blogs, and allow administrators to dynamically control website content through a secure Content Management System (CMS).

This document serves as a master reference for:

- System development
- Technical documentation
- Project proposal or defense
- Team onboarding

2 System Architecture

The system follows a modern, scalable, production-ready architecture.

```
[ Frontend (Next.js + Tailwind) ]
            |
         REST API
            |
[ Backend (Node.js + TypeScript) ]
            |
[ Admin Portal (CMS Dashboard) ]
            |
[ Database (PostgreSQL / MySQL) ]
```

Architecture Description

- Frontend: Public-facing website for visitors
- Admin Portal: Secure CMS for content management
- Backend API: Business logic, authentication, and data access
- Database: Persistent storage

3 Branding and UI Design System (Tailwind)

3.1 Color Palette

Role	Color
Primary	#7B1E1E (Shaolin Red)
Dark	#111111
Accent	#C9A24D (Heritage Gold)
Text	#FFFFFF
Muted	#9CA3AF

3.2 Tailwind Configuration

```
extend: {
  colors: {
    primary: '#7B1E1E',
    dark: '#111111',
    gold: '#C9A24D',
  }
}
```

3.3 Typography

Usage	Font
Headings	Playfair Display (Serif)
Body Text	Inter / Poppins (Sans-serif)

4 Frontend Website (Public)

4.1 Design Goals

- Cinematic visuals
- Martial arts strength and discipline
- Traditional yet modern appearance
- Fully CMS-driven content
- Mobile-first responsiveness

5 Frontend Pages and Sections

5.1 Header and Navigation

- Logo
- Navigation: Home, About, Blog, Clubs, Contact
- Language switcher (RW / EN / FR)

CMS Control: None (static navigation)

5.2 Hero Section

- Full-screen background image
- Dark overlay
- Title, subtitle, CTA button
- Carousel slider

CMS Controlled:

- Title and subtitle
- Background images
- Button text and link

5.3 About Us Section

- Image and text layout
- Founder quotation

CMS Controlled:

- About text
- Founder name
- Images
- Year founded

5.4 Programs and Training

- Online Training
- In-Person Training
- Global Retreats

Each program contains:

- Image
- Title
- Description
- Status (active or inactive)

5.5 Blogs

- Featured blog
- Blog listing
- Blog detail page

5.6 Clubs

- Club name
- Location
- Instructor
- Description
- Images

5.7 Contact Us

- Name
- Email
- Message

5.8 Gallery

- Photo and video media
- Categorization by event
- Date and title metadata

Backend Requirements:

- Message storage
- Admin replies
- Email notifications

6 Admin Portal (CMS Dashboard)

6.1 Authentication and Roles

Role	Permissions
Super Admin	Full access
Content Manager	Create and edit content
Editor	Limited editing

6.2 Dashboard Modules

- Dashboard overview and system statistics
- Homepage (Hero Slides) manager
- About manager
- Programs manager
- Blog manager
- Clubs and Tutorials manager
- Gallery (Media) manager
- Messages manager
- Audit Logs (Activity tracking)
- Language manager
- Users and roles

7 Backend (API and Logic)

7.1 Core Responsibilities

- Authentication and authorization using JWT
- Content APIs
- Image uploads
- Email services
- Role-based access control

7.2 Technology Stack

Backend:

- Node.js with Express
- JWT authentication and Authorization middleware
- Multer and Cloudinary for media storage
- Nodemailer for email invitations and password resets
- Swagger (OpenAPI) for interactive API documentation
- Audit Logging for administrative actions

Database:

- PostgreSQL or MySQL
- Prisma or TypeORM

8 Database Models

```
User(id, name, email, password, role, image, reset_token,
   created_at)

HeroSlide(id, title, subtitle, image, button_text, button_link)

About(id, content, founder_name, image)

Program(id, name, description, image, status)
```

```
Blog(id, title, content, image, category, published, created_at)

Club(id, name, description, location, image)

ClubTutorial(id, club_id, title, video_url)

Gallery(id, title, description, url, media_type, event_date,
   category)

ContactMessage(id, name, email, message, read, created_at)

AuditLog(id, user_id, action, entity, entity_id, details,
   ip_address)
```

9 Advanced System Features

9.1 Audit Logging

All administrative actions (Create, Update, Delete, Restore) are logged in the system. Each log entry captures:

- The administrator who performed the action
- The type of action and the entity affected
- A timestamp and the administrator's IP address

9.2 Data Recovery and Soft Deletes

The system implements soft deletes (paranoid mode), allowing administrators to restore accidentally deleted content. Permanent deletion is only possible through a deliberate "Force Delete" action.

9.3 Interactive API Documentation

The backend includes a Swagger (OpenAPI) interface, providing real-time documentation and a testing environment for all API endpoints.

9.4 ID Obfuscation (UUID Encryption)

To enhance security and prevent exposure of internal database structure, the system implements ID obfuscation using Base64URL encoding:

- All public-facing UUIDs are encoded using Base64URL (URL-safe encryption)
- Backend transparently decodes obfuscated IDs via middleware
- Prevents enumeration attacks and information disclosure
- Maintains backward compatibility with existing UUID structure

Implementation:

```
// Encoding: UUID -> Base64URL
const encodeUUID = (uuid) => {
  const hex = uuid.replace(/-/g, '');
  const buffer = Buffer.from(hex, 'hex');
  return buffer.toString('base64')
    .replace(/\+/g, '-')
    .replace(/\//g, '_')
    .replace(/=/g, '');
};

// Decoding middleware applied to all routes
router.get('/:id', decodeParam('id'), controller.getById);
```

9.5 Database Schema Standardization

All Sequelize models have been standardized with explicit $snake_case$ timestamp mappings to ensure database consistency:

- Explicit mapping: `createdAt:` '$created_at$', `updatedAt:` '$updated_at$'
- Resolves Sequelize ordering and query inconsistencies
- Prevents 500 Internal Server Errors from column name mismatches
- Applied to all models: Club, Program, Gallery, About, HeroSlide, User, Blog, ContactMessage, AuditLog

9.6 Frontend Stability Enhancements

The admin portal includes robust error handling and defensive programming practices:

- Optional chaining for safe property access (`pagination?.totalItems`)
- Default values for undefined data states
- Pagination protection against crashes during network latency
- Error boundaries for graceful degradation

10 Non-Functional Requirements

- Mobile-first responsive design
- Search engine optimization
- Secure admin routes
- Optimized images
- Fast loading performance

11 Final Summary

The Kigali Shaolin Temple Management System consists of three integrated layers:

Layer	Purpose
Frontend	Public Shaolin Temple website
Admin Portal	CMS for content management
Backend	API, authentication, database

This design:

- Covers the majority of UI requirements from the provided design
- Uses Tailwind CSS for a modern and professional interface
- Enables scalability through CMS control